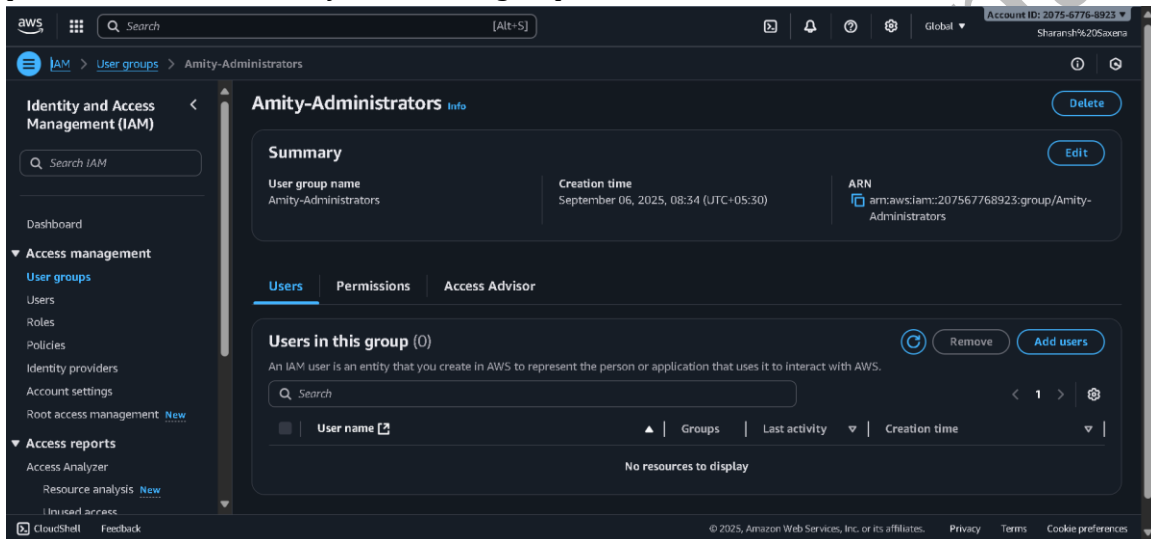


Applied Cloud Computing – DIY Project 1

This document contains the step-by-step execution of AWS IAM and EC2 tasks as part of DIY Project 1 for Applied Cloud Computing. Each task includes explanation, commands and screenshots.

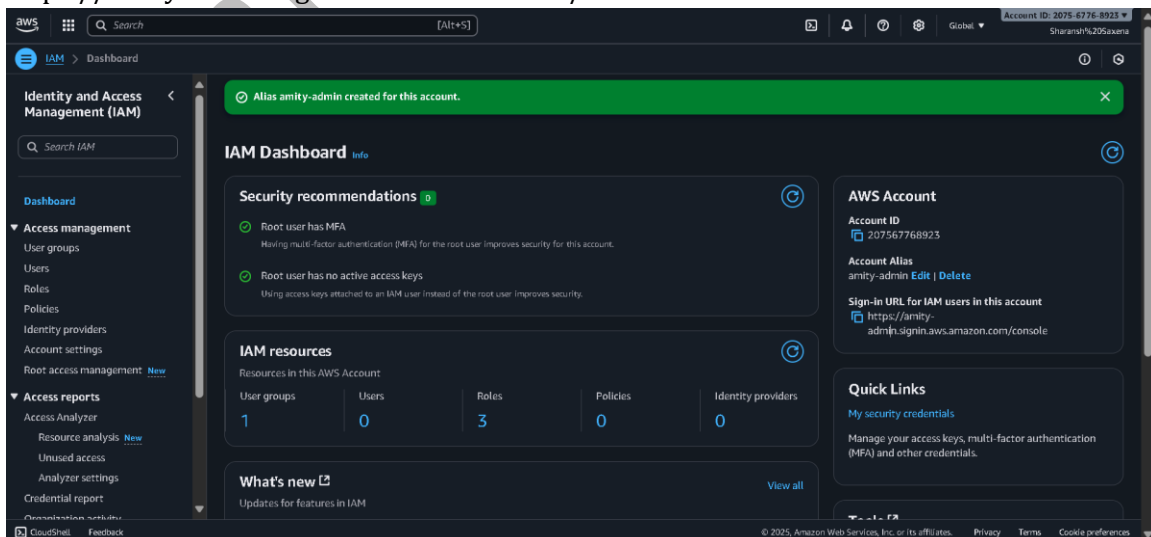
Step 1: Create IAM Group

Login as root user and create an IAM group called Administrators. Attach the managed policy 'Amity-Administrators' to this group. Since AWS reserves the name **Administrator** for internal use, I created a group named **Amity-Administrators** with the same permissions and added my user to this group.



Step 2: Customize Sign-in Link

Customize a sign-in link for your AWS account and note the new link. Example:
<https://amity-admin.signin.aws.amazon.com/console>



Applied Cloud Computing – DIY Project 1

Step 3: Create Password Policy

Create a strong password policy for your AWS account. Set requirements such as minimum length, uppercase, numbers, and symbols.

The screenshot shows the 'Edit password policy' page in the AWS IAM console. The 'Custom' radio button is selected under the 'Password policy' section. The 'Password minimum length' is set to 8 characters. Under 'Password strength', the following requirements are checked: 'Require at least one uppercase letter from the Latin alphabet (A-Z)', 'Require at least one lowercase letter from the Latin alphabet (a-z)', 'Require at least one number', and 'Require at least one non-alphanumeric character (!@#\$%^&*()_+-=[]{}|)'. Under 'Other requirements', 'Turn on password expiration', 'Password expiration requires administrator reset', 'Allow users to change their own password', and 'Prevent password reuse' are all checked. The 'Save changes' button is highlighted in orange at the bottom right.

Step 4: Create IAM User

While logged in as root user, create a new IAM user called 'Administrator' and add this user to the Administrators group.

Since the Review page could not fit in one screenshot, I have captured it in two parts.

The screenshot shows the 'Review and add user' page in the AWS IAM console, specifically 'Step 1: Specify user details'. The 'Primary information' section contains the following details:

Attribute key	Value
Username	Administrator1
Email	sharanshaxena39959@gmail.com
First name	Sharansh
Last name	Saxena
Display name	Administrator1

Below the primary information, there are four expandable sections for optional information: 'Contact methods - optional', 'Job-related information - optional', 'Address - optional', and 'Preferences - optional'. The 'Edit' button is located at the top right of the 'Primary information' section.

Applied Cloud Computing – DIY Project 1

This screenshot shows the 'Add user' wizard in the AWS IAM console, specifically Step 2: 'Add user to groups - optional'. The user 'Administrator1' has been created. The wizard offers optional configurations for contact methods, job-related information, address, preferences, and additional attributes. Below these, there is a section to add the user to groups. A table shows one group, 'Admin-Administrator', which is selected. The 'Add user' button is highlighted in orange.

Display name: Administrator1

► Contact methods - optional

► Job-related information - optional

► Address - optional

► Preferences - optional

► Additional attributes - optional

Step 2: Add user to groups - optional [Edit](#)

Group name	Description
Admin-Administrator	-

[Cancel](#) [Previous](#) [Add user](#)

This screenshot shows the 'Users' list page in the AWS IAM console. It displays a single user, 'Administrator1', which is enabled and visible. The table includes columns for Username, Display name, Status, MFA devices, and Created by. The 'Status' column shows 'Enabled' with a green checkmark. The 'Created by' column shows 'Manual'.

Users (1) [Delete users](#) [Add user](#)

Users listed here can sign in to the AWS access portal to access AWS accounts and assigned cloud applications. [Learn more](#)

Username	Display name	Status	MFA devices	Created by
Administrator1	Administrator1	Enabled	None	Manual

Users list showing Administrator1 enabled and visible.

Applied Cloud Computing – DIY Project 1

The image displays two screenshots of the AWS IAM console interface, showing the details of a user named Administrator1.

Top Screenshot: Groups Tab

- User:** Administrator1
- Status:** Email not verified (Warning icon). Users must first verify their email address before they can begin to use certain features such as completing email-based two-step verification during sign-in.
- Buttons:** Reset password, Delete user, Send email verification link, Disable user access.
- General information:** Profile, Groups (1), AWS accounts, Applications, MFA devices (0), Active sessions (0).
- Group memberships (1):** To grant permission to this user, add the user to a group that has access to AWS accounts or cloud applications. [Learn more](#)
- Search:** Search groups by group name
- Table:**

Group name	Description
Amity-Administrator	-

Bottom Screenshot: Profile Tab

- User:** Administrator1
- Status:** Email not verified (Warning icon). Users must first verify their email address before they can begin to use certain features such as completing email-based two-step verification during sign-in.
- Buttons:** Reset password, Delete user, Send email verification link, Disable user access, Edit.
- Profile details:** Search attribute
- Primary information:**

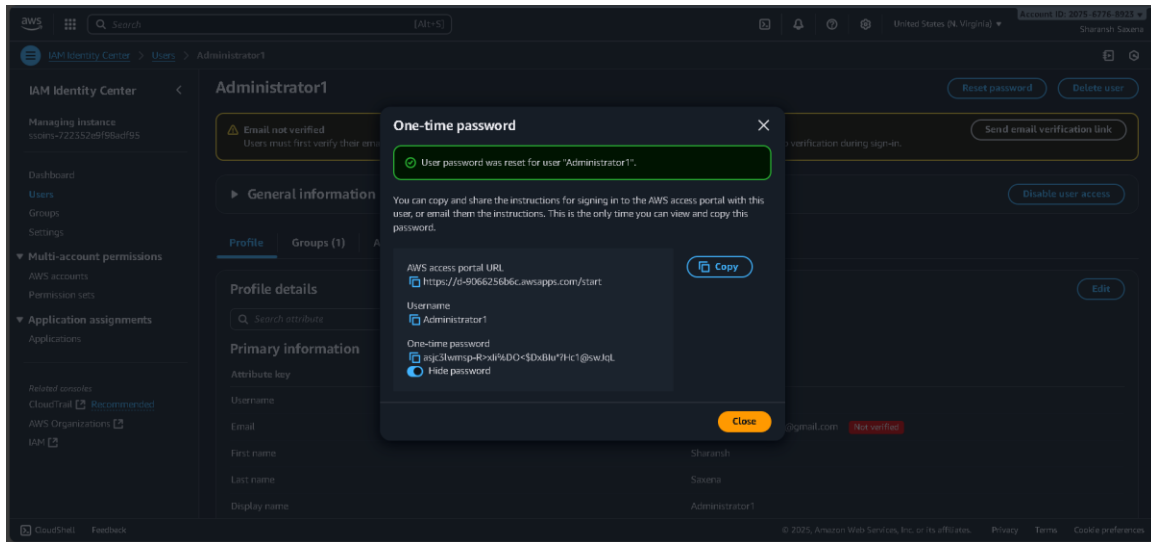
Attribute key	Value
Username	Administrator1
Email	sharamshaxena39090@gmail.com Not verified
First name	Sharansh
Last name	Saxena
Display name	Administrator1

User details page of Administrator1, showing Group memberships section with Amity-Administrators.

Applied Cloud Computing – DIY Project 1

Step 5: Create Password for IAM User

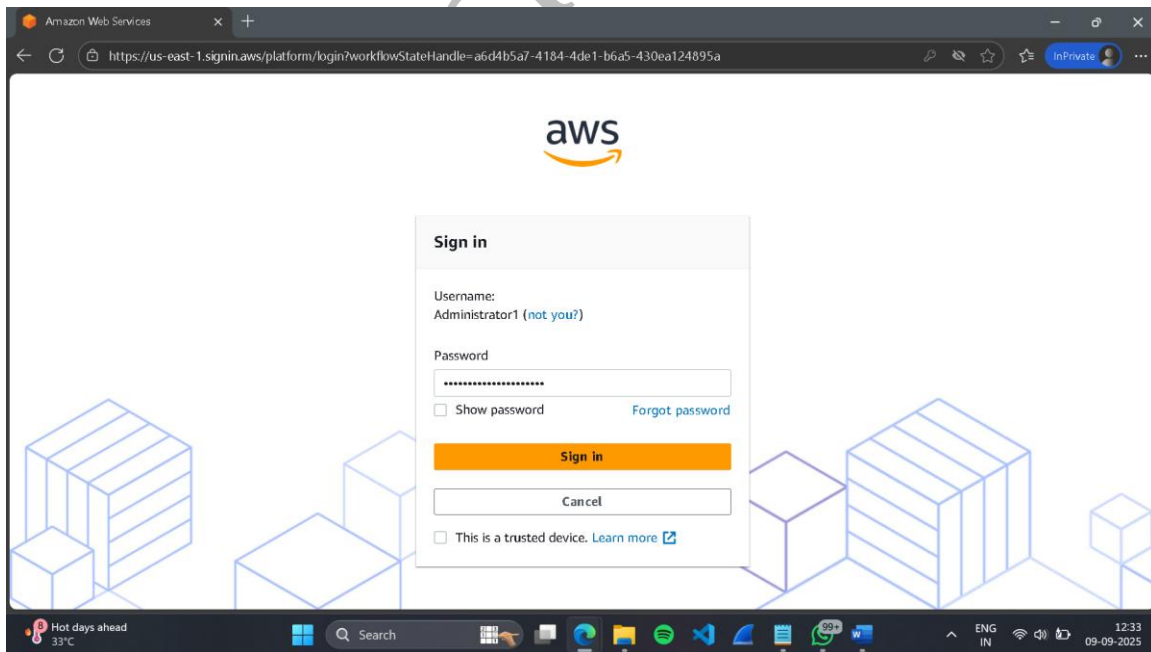
On the details page for the Administrator user, set a password for console access.



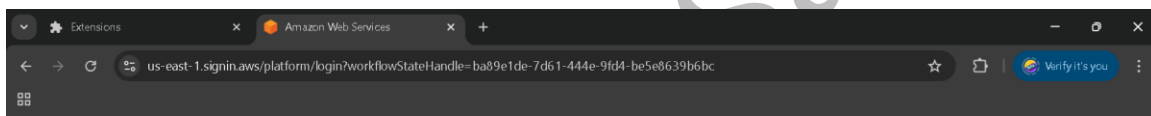
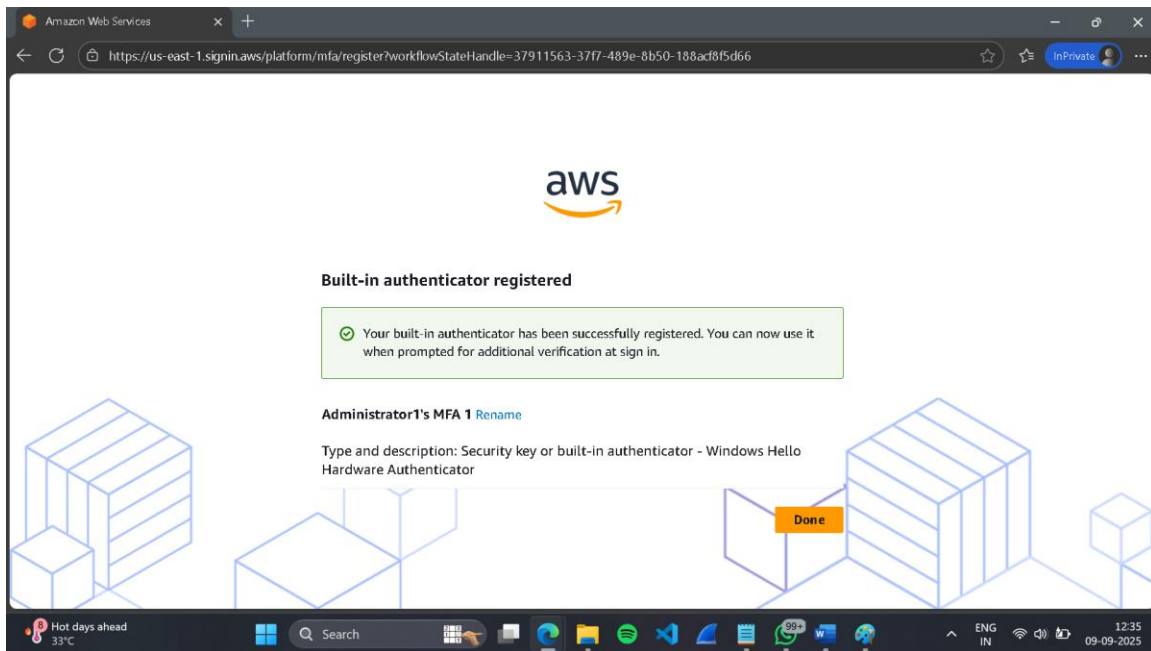
Confirmation page showing AWS Access Portal URL, Username (Administrator1), and the generated one-time password.

Step 6: Login as Administrator

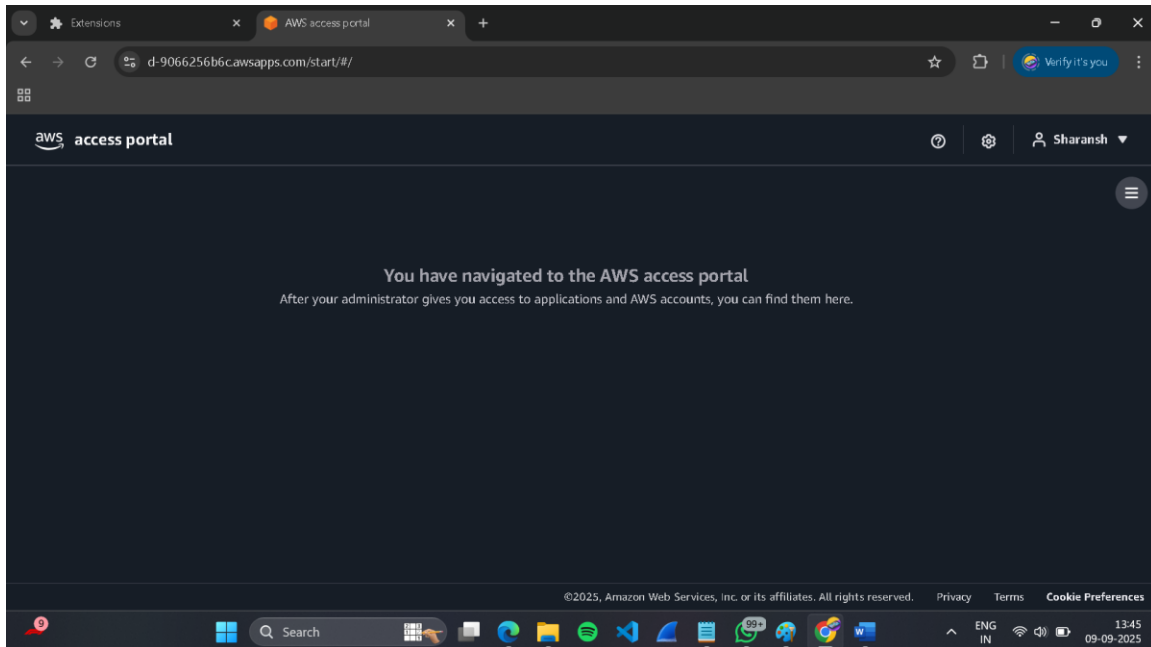
Log out as root user and log in with the customized sign-in link using the Administrator user credentials.



Applied Cloud Computing – DIY Project 1



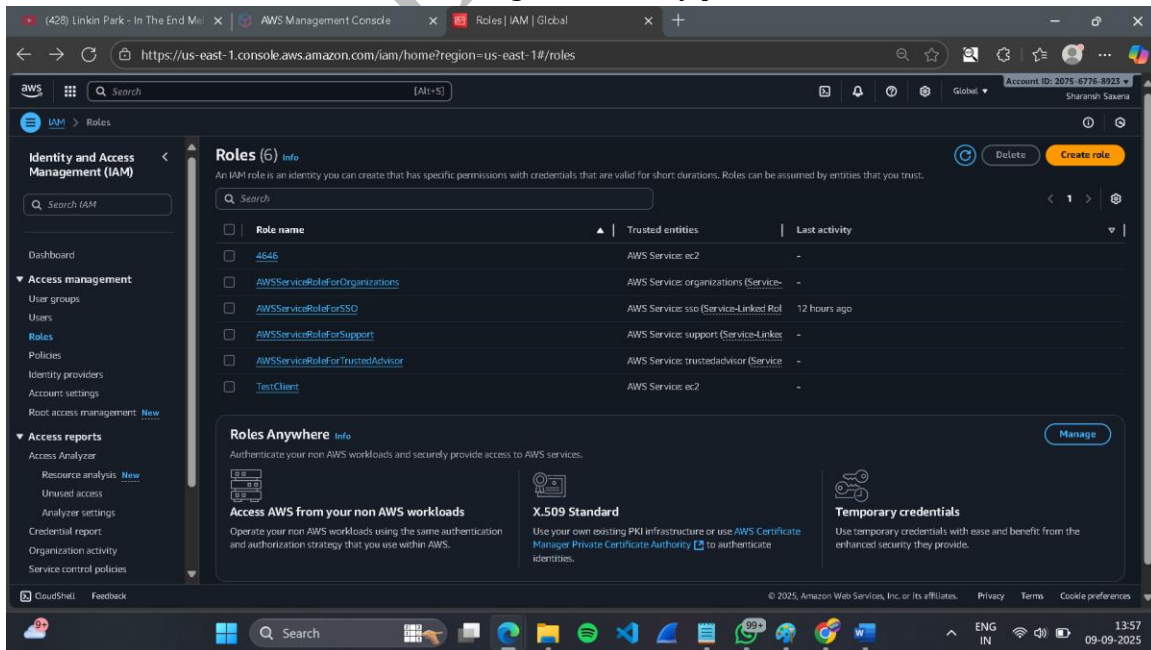
Applied Cloud Computing – DIY Project 1



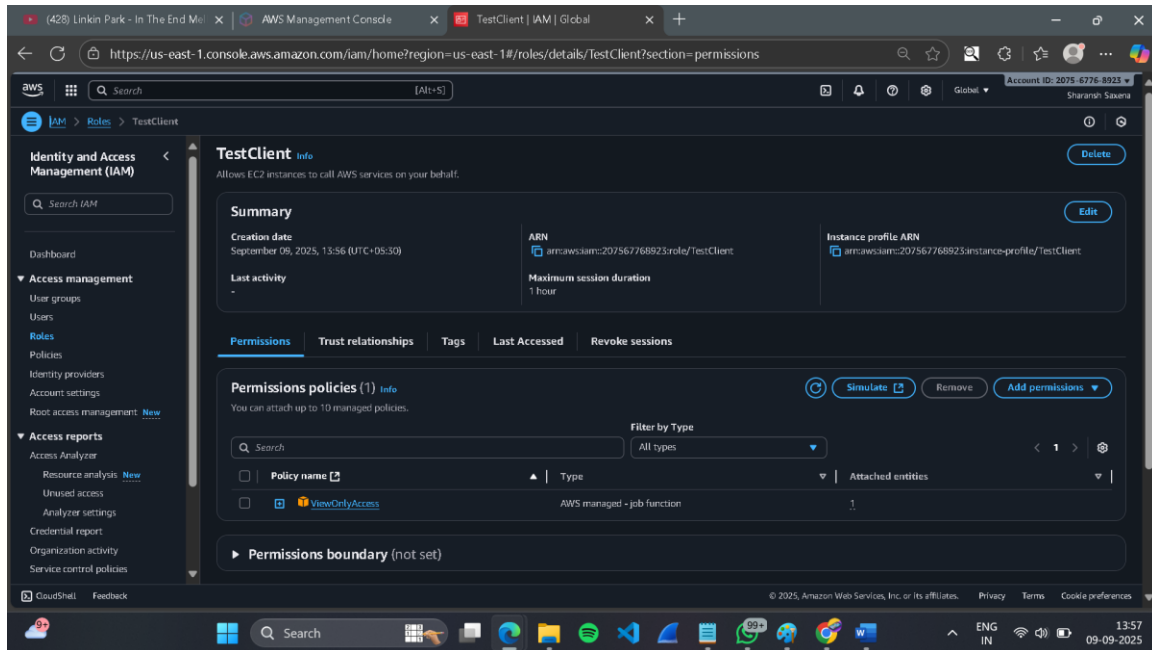
The above screenshot shows the successful login of the newly created user Administrator1 through the customized AWS Access Portal sign-in link. After entering the one-time password and resetting it to a new strong password, the user was able to access the AWS Access Portal dashboard.

Step 7: Create IAM Role for EC2

While signed in as the Administrator, create an IAM role named **TestClient** with trusted entity type **EC2**. Attach the AWS managed policy **ViewOnlyAccess** to this role. This allows the EC2 instance to assume the role and gain view-only permissions for AWS resources.



Applied Cloud Computing – DIY Project 1

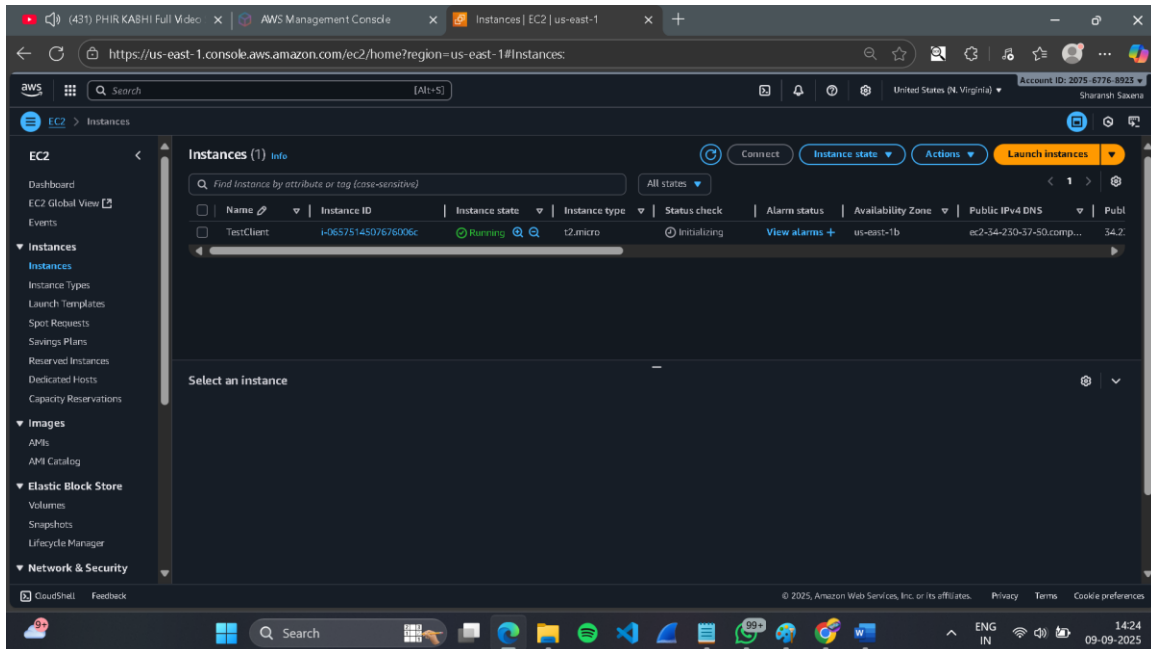


The IAM role “TestClient” was created with EC2 as the trusted entity. Since the “ReadOnlyAccess” policy was not available in my AWS account, I attached the AWS managed policy “ViewOnlyAccess.” This policy provides equivalent view-only permissions to AWS resources and fulfills the assignment requirement.

Applied Cloud Computing – DIY Project 1

Step 8: Launch EC2 Instance

Launch an Amazon Linux EC2 instance and attach the 'TestClient' role to it.



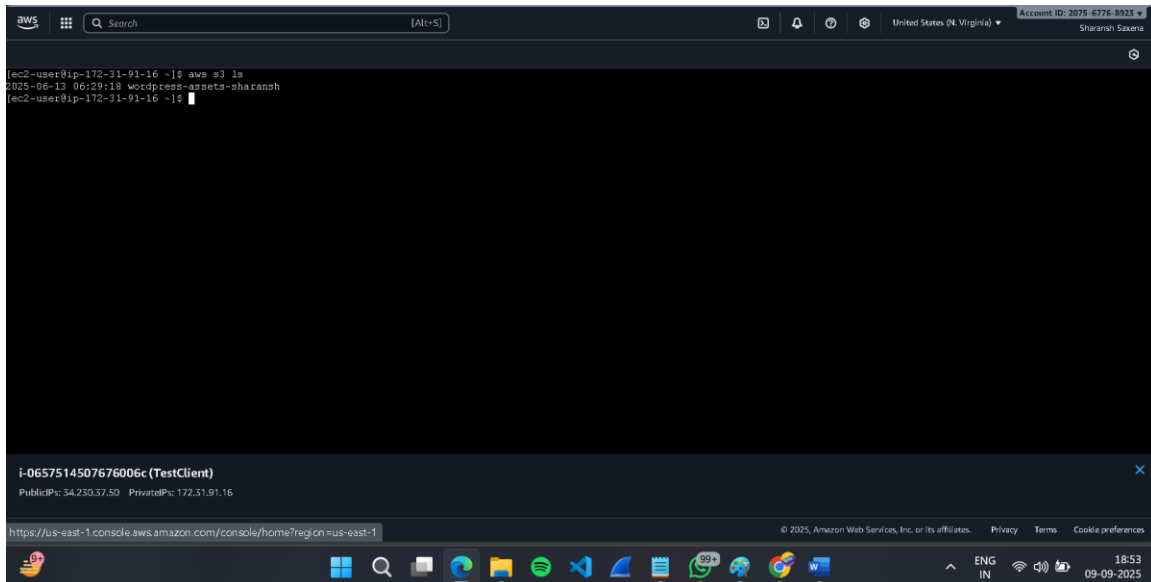
The above screenshot shows the EC2 instance “TestClient (Instance ID: i-0657514507676006c)” running in Availability Zone us-east-1b. The instance is of type t2.micro, launched with the key pair “TestClient-Key” and attached to the IAM role “TestClient”. The public IPv4 address 34.230.37.50 will be used to connect via SSH.

Applied Cloud Computing – DIY Project 1

Step 9: SSH into EC2 & List S3 Buckets

SSH into the EC2 instance and run the following command to list S3 buckets:

```
aws s3 ls
```



The above screenshot shows the EC2 instance successfully accessing Amazon S3 using the IAM role TestClient. The command `aws s3 ls` lists the available bucket `wordpress-assets-sharanish`, verifying that the role permissions are working correctly.

Applied Cloud Computing – DIY Project 1

Step 10: Create Conflicting Policy

Use the policy generator to create a new policy with the following details:

- Effect: Deny
- Service: Amazon S3
- Actions: *
- ARN: *

Attach this policy to the Administrators group.

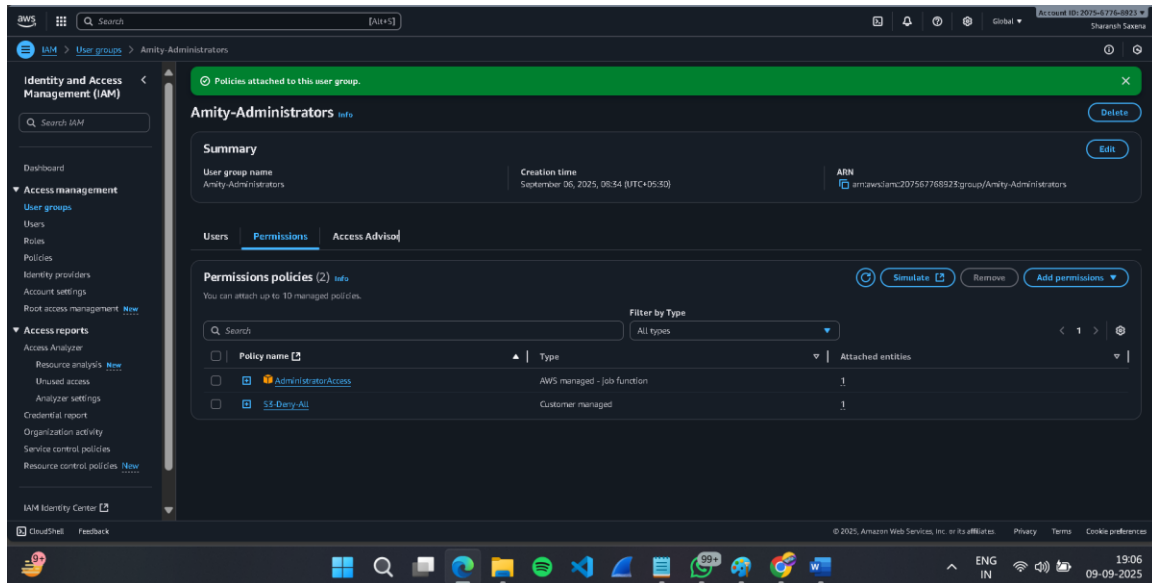
The first screenshot shows the 'Specify permissions' step in the AWS IAM console. A red error message states: 'No actions specified. No actions are specified. Specify at least one action for the selected services.' The 'Policy editor' is in JSON mode, displaying the following code:

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Deny",  
6       "Action": "*",  
7       "Resource": "*" }  
8   ]  
9 }  
10  
11
```

The second screenshot shows the 'S3-Deny-All' policy details page. The policy is customer managed, created on September 08, 2025, at 19:01 UTC+05:30. The ARN is `arnawsiam:207567768023:policy/S3-Deny-All`. Under the 'Permissions defined in this policy' section, it shows 'Explicit deny (1 of 450 services)' for the S3 service with full access across all resources and no request conditions. The 'Allow (0 of 450 services)' section is empty.

The above screenshot shows the custom IAM policy S3-Deny-All created to explicitly deny all Amazon S3 actions across all resources. This policy is customer managed and will be attached to the Amity-Administrators group to test the Deny precedence over Allow policies.

Applied Cloud Computing – DIY Project 1



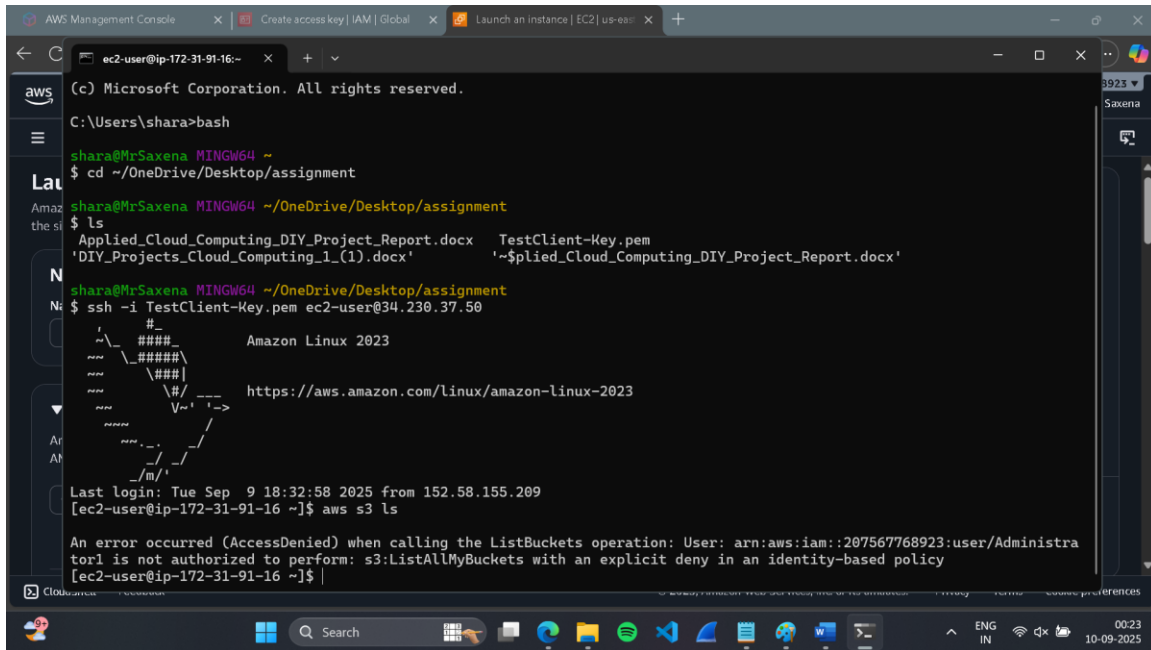
The above screenshot shows the custom IAM policy S3-Deny-All created using the JSON editor. This policy explicitly denies all Amazon S3 actions across all resources and will later be attached to the Amity-Administrators group to test Deny precedence.

Applied Cloud Computing – DIY Project 1

Step 11: Test Access Denial

Use the AWS CLI again to try listing S3 buckets. Even though the 'ReadOnlyAccess' policy allows access, the new Deny policy overrides it, so access is denied.

aws s3 ls



```
(c) Microsoft Corporation. All rights reserved.
C:\Users\shara>bash
shara@MrSaxena MINGW64 ~
$ cd ~/OneDrive/Desktop/assignment
shara@MrSaxena MINGW64 ~/OneDrive/Desktop/assignment
$ ls
Applied_Cloud_Computing_DIY_Project_Report.docx  TestClient-Key.pem
'DIY_Projects_Cloud_Computing_1_(1).docx'        '~$plied_Cloud_Computing_DIY_Project_Report.docx'
shara@MrSaxena MINGW64 ~/OneDrive/Desktop/assignment
$ ssh -i TestClient-Key.pem ec2-user@34.230.37.50
#
      _ _ _ _ _
     / _ _ _ _ \   Amazon Linux 2023
    / _ _ _ _ \
   / _ _ _ _ \
  / _ _ _ _ \
 / _ _ _ _ \
/_ _ _ _ _ \
/m/

Last login: Tue Sep  9 18:32:58 2025 from 152.58.155.209
[ec2-user@ip-172-31-91-16 ~]$ aws s3 ls

An error occurred (AccessDenied) when calling the ListBuckets operation: User: arn:aws:iam::207567768923:user/Administrator1 is not authorized to perform: s3:ListAllMyBuckets with an explicit deny in an identity-based policy
[ec2-user@ip-172-31-91-16 ~]$
```

The above screenshot shows the EC2 instance user Administrator1 attempting to list S3 buckets. Due to the attached S3-Deny-All policy, the operation was denied with an explicit AccessDenied error. This validates that Deny policies override any Allow permissions in AWS IAM.